ByteDance | Seed

Published as a conference paper at ICML 2025

Understanding Chain-of-Thought in LLMs through Information Theory

Jean-Francois Ton[*,1], **Muhammad Faaiz Taufiq**[*,1], **Yang Liu**[2]

[1]ByteDance Seed, [2]University of California, Santa Cruz

[*]All authors contributed equally; ordering determined through a coin flip.

arXiv:2411.11984v2 [cs.CL] 10 Jul 2025

Abstract

Large Language Models (LLMs) have shown impressive performance in complex reasoning tasks through the use of Chain-of-Thought (CoT) reasoning, allowing models to break down problems into manageable sub-tasks. However, existing CoT evaluation techniques either require annotated CoT data or fall short in accurately assessing intermediate reasoning steps, leading to high rates of false positives. In this paper, we formalize CoT reasoning in LLMs through an information-theoretic lens. Specifically, our framework quantifies the 'information-gain' at each reasoning step, enabling the identification of failure modes in LLMs without the need for expensive annotated datasets. We demonstrate the efficacy of our approach through extensive experiments on toy arithmetic, GSM8K and PRM800k datasets, where it significantly outperforms existing outcome-based methods by providing more accurate insights into model performance on individual subtasks.

Correspondence: jeanfrancois@bytedance.com, faaiz.taufiq@bytedance.com

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of tasks, from complex reasoning to code generation [2–4, 23]. Many of these advances can be attributed to Chain-of-Thought (CoT) reasoning [17, 22, 32], which involves breaking down complex problems into a series of intermediate steps, mirroring human-like reasoning processes. The success of CoT reasoning, particularly in domains such as mathematics, logic, and multi-step decision-making, has led researchers to incorporate CoT-like features directly into model training, i.e. the FLAN family of models [5, 31].

This paper introduces a new formal framework for analyzing CoT in LLMs. We provide a rigorous method grounded in information theory, to evaluate the quality of each step in a model's reasoning process, thus offering insights beyond simple accuracy metrics to identify areas for improvement.

Previous work in this area has proposed "*Process Supervision*" [18], which requires expensive, human-annotated step-by-step data. While effective, this approach is often impractical due to the high cost and effort of creating large-scale annotated datasets. In turn, alternative methods have recently been proposed, such as outcome reward modelling [11] or the Math-Shepherd [29]. Both these approaches avoid reliance on costly annotated step-wise CoT data by instead modelling the correctness of each step based on the correctness of final outputs. However, as we show later, these methods can be unsound for detecting incorrect reasoning steps and can thus lead to a high false-positive rate in certain scenarios.

To address these shortcomings, we employ an information-theoretic approach, grounded in the following key

insight: *Each correct step in a reasoning process should provide valuable and relevant information that aids in predicting the final correct outcome.* Building on this insight, we develop a framework to quantify the "*information-gain*" after each sub-task in the reasoning process, without the need for step-by-step annotations. This enables us to detect sub-tasks that fail to contribute meaningful information toward the correct solution, signalling potential errors or irrelevant steps in the model's reasoning. In addition, we also introduce a practical algorithm to assess LLM performance across various sub-tasks within a Chain-of-Thought (CoT) reasoning process. The key contributions of this paper are as follows:

1. We develop a framework for sequential applications of sub-tasks, e.g. Chain-of-Thought and provide a rigorous language to identify failure modes in LLMs.
2. Based on this framework, we propose a practical algorithm to assess the task-wise performance of models. This yields more granular information about a model's CoT performance without requiring annotated data for intermediate reasoning steps.
3. We validate our methods on extensive toy data, the GSM8K [6] as well as the PRM800K [18] dataset. Our method effectively identifies failure modes in CoT reasoning, unlike baselines like outcome reward modelling [11] and Math-Shepherd [29], which rely on final accuracy and tend to increase false positives in error detection.

2 Proposed Framework: Setup and Notation

Before diving into our framework, we first provide a high-level overview and notation on how LLM generation will be treated throughout this paper. This will allow us to set the foundation for describing our information-theoretic framework. In particular, following the approach in [10], we view LLMs as abstract execution machines with a natural language interface. From this perspective, prompts are designed to solve specific problems (e.g., mathematical or logical problems), and the LLM processes the information in the prompt to generate an output.

We define a typical prompt as a combination of two parts:

1. An initial state, represented by the random variable $X_0 \in \mathcal{X}$, encapsulates the prompt-provided information that the LLM processes to derive the queried result.
2. A task $\lambda \in \Upsilon$ (e.g., addition then multiplication) defines how the LLM processes X_0.

Given the prompt, defined as a tuple (X_0, λ), the state X_1 represents the result of applying task λ to the initial state X_0. Formally, we denote this using the <u>update</u> mapping $\Lambda : \mathcal{X} \times \Upsilon \to \mathcal{X}$ which outputs the updated state X_1 by applying the task λ on X_0, i.e. $X_1 = \Lambda(X_0, \lambda)$. This updated state is then used to obtain the final output, denoted by $Y \in \mathcal{X}$, by extracting only the information in X_1 which is relevant to the queried final answer. This notation defines a prompt that instructs a model to process information drawn from some initial distribution $p(X_0)$ (e.g., math problems).

We use a simple example to illustrate this notation:

$$\textbf{Prompt: } \textit{"Solve for } z = 2\,(u+v),\ u = 12,\ v = 13\textit{"} \tag{1}$$

Here, the initial state x_0 denotes the information *"u = 12, v = 13"*, and λ denotes the task of finding z (i.e. addition followed by multiplication). Next, $x_1 = \Lambda(x_0, \lambda)$ represents the updated information after correctly performing the addition operation, i.e. x_1 is *"u = 12, v = 13 and z = 50"*. The final output, y, is then obtained by simply extracting the value of z from x_1, i.e. *"z = 50"*.

Remark Our setup also encapsulates cases with ambiguous (or multiple correct) responses for a given task λ. In this case, $\Lambda(x_0, \lambda)$ is a random variable with distribution $p(X_1 \mid X_0 = x_0)$. Therefore, for generality, we treat $\Lambda(x_0, \lambda)$ as a random variable from now on.

2.1 Compositionality

Many mathematical or logical problems, such as the one in (1), require sequential application of operations. Our notation is also amenable to such problems as it accommodates the composition of tasks.

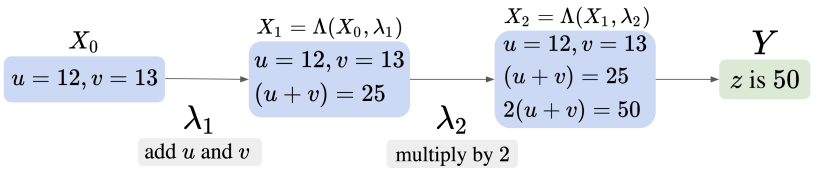

Figure 1 Prompt (1) requires compositional application of tasks.

For example, one way to address prompt (1) involves first adding u and v, and next, multiplying the result by 2 to find z. Using our notation, this can be expressed as $\Lambda(x_0, \lambda_1 \circ \lambda_2)$, where λ_1, λ_2 denote the addition and multiplication tasks respectively. The following property allows us to define the application of compositional task $\lambda_1 \circ \lambda_2$:

Definition 2.1. We say that an update rule $\Lambda : \mathcal{X} \times \Upsilon \to \mathcal{X}$ is compositionally consistent if, for all $x_0 \in \mathcal{X}$ and $\lambda_1, \lambda_2 \in \Upsilon$ we have that $\Lambda(x_0, \lambda_1 \circ \lambda_2) \overset{\text{d}}{=} \Lambda(\Lambda(x_0, \lambda_1), \lambda_2)$.

Here, $\overset{\text{d}}{=}$ denotes equality in distribution and is sufficient in cases where a query may have multiple correct responses.

Returning to the prompt in (1), Figure 1 shows that the model first computes $u + v$, then multiplies the result by 2. Here, we refer to X_1, X_2 as intermediate states and Y is the correct final output. In general, if a problem statement requires sequential application of T sub-tasks, $\lambda = \lambda_1 \circ \ldots \circ \lambda_T$, then the Chain-of-Thought (CoT) reasoning is divided up into T steps, where the t'th step is recursively defined as $X_t = \Lambda(X_{t-1}, \lambda_t)$ for $t \in \{1, \ldots, T\}$. Finally, the overall true output Y is obtained by extracting the queried information from the final state X_T.

Having established a formal language for the sequential application of tasks, we now turn towards how a task may be divided into such a sequence of intermediate sub-tasks.

2.2 Primitive Tasks

In this subsection, we introduce the notion of primitive tasks which form the basic building blocks of any task. Intuitively, our formulation is reminiscent of ideas from linear algebra, where basis vectors form the basic building blocks of a vector space. In our case, any task $\lambda \in \Upsilon$ can be expressed as a sequence of primitive tasks. This decomposition will allow us to establish which tasks the model could have learned from the training data. For example, if a specific primitive task is not available in the LLM training data, it would be impossible for the model to execute any instructions which involve this primitive task correctly. With this in mind, we now introduce this concept formally:

Definition 2.2 (Primitive tasks)**.** We say that a set of tasks $\Gamma \subseteq \Upsilon$ is primitive if, for any task $\lambda \in \Upsilon$, there exists a unique subset $\{\lambda_i\}_{i=1}^k \subseteq \Gamma$ such that $\lambda = \lambda_1 \circ \cdots \circ \lambda_k$.

Note that the decomposition is not unique but the set of components is. In some cases, there may exist distinct permutations of primitive tasks which compose to yield the same task as is common in many associative operations. As an example, in the context of mathematical problem-solving, the basic arithmetic operations could be considered primitive. The composition of these primitive tasks allows us to construct extremely complex operations. Just like in linear algebra, we define the span of these tasks as the set obtained by their sequential applications.

Definition 2.3 (Span of tasks)**.** Let $\Phi \subseteq \Upsilon$ be a set of tasks:

$$\text{Span}(\Phi) = \{\lambda_1 \circ \ldots \circ \lambda_k : \lambda_i \in \Phi \text{ for } 1 \leq i \leq k, k \in \mathbb{Z}_{>0}\}.$$

The set $\text{Span}(\Phi)$ comprises all the tasks that can be applied by composing sub-tasks in the set Φ. This means that any compositionally consistent update rule Λ which is well-defined on the set of tasks Φ will also be well-defined on $\text{Span}(\Phi)$. However, this Λ may still be ill-defined for any task not in this span. This limitation, known as unidentifiability, defines the boundaries of a model's inferences.